

Descrizione UML

Model

(Modifiche rispetto alla versione precedente)

Rimozione di GameManager & implementazioni:

L'interfaccia di classe GameManager è stata rimossa in quanto la sua funzionalità è ora completamente gestita mediante la classe controller GameManagerController.

Introduzione interfaccia Shelf & implementazioni:

L'interfaccia definisce i metodi per gestire la rappresentazione della shelf di un singolo giocatore.

Trasformazione dell'interfaccia Chat in PlayerChat

L'interfaccia di Chat rimane invariata, ma ne viene modificata l'interpretazione: l'interfaccia è ora indirizzata alla classe che gestisce i messaggi di un singolo giocatore; sarà compito del controller quello di smistare l'invio/ricezione dei messaggi.

Astrazione interfaccia Game & implementazioni

La classe Game è stata suddivisa in interfaccia & implementazione, l'interfaccia definisce ora tutti i metodi necessari a gestire e ottenere le informazioni/oggetti che compongono lo schema di gioco.

Rimozione PlayerStatus

La classe PlayerStatus è stata rimossa in quanto la sua funzionalità è assegnata alle implementazioni dell'interfaccia GameController.

Astrazione interfaccia Player & implementazioni

La classe Player è stata suddivisa in interfaccia & implementazione, sono stati inoltre ridefiniti i metodi che offre, sulla base delle modifiche alla interpretazione dell'oggetto "giocatore", in particolare:

- Il giocatore non possiede più una interpretazione dei punti guadagnati dal conseguimento di CommonGoal attraverso un punteggio, ma possiede ora una mappa che associa ad ogni CommonGoal il Token guadagnato.
- `getShelf()` ora ritorna una interfaccia Shelf.
- `getPlayerChat()` ritorna la Chat (insieme dei messaggi) riguardanti il giocatore
- `reportError()` permette di segnalare che il giocatore ha riportato un errore durante un'azione di gioco (messaggio malformato, mossa illegale etc.).

Introduzione della classe User

La classe User identifica a livello di Model la presenza di un utente collegato al server, User possiede uno stato definito dall'enumerabile User.Status e un messaggio di errore che può essere assegnato quando l'utente riporta un errore durante un'azione sul server (ingresso ad una partita, creazione di una partita etc.).

Estensione delle interfacce del model della classe Observable

Le interfacce User, Game, CommonGoal, LivingRoom, Player, Shelf, PersonalGoal e PlayerChat estendono ora la classe Observable, sono dunque definite non più come interfacce ma come classi astratte. Per ogni classe astratta sono stati inoltre definiti alcuni eventi corrispondenti in enumerabili *.Event .

Util

Il package Util contiene le classi di supporto che implementano specifiche funzionalità (nel nostro caso il solo pattern Observer-Observable).

Observer<> - Observable<>

Le classi Observer<> e Observable<> sono le ridifinizioni su Java Generics con supporto al multi-thread delle classi Observer e Observable.

Controller

Il package Controller è stato completamente ridefinito sulla base di una nuova interpretazione delle classi:

<Interface> ServerController

Questa interfaccia definisce metodi con i quali è possibile interagire con l'istanza "Server" al fine di creare una partita e accedere ad una partita.

<Interface> GamesManagerController

Questa interfaccia definisce metodi con i quali è possibile creare e ottenere il controllo di una partita.

<Interface> GameController

Questa interfaccia definisce i metodi con i quali è possibile interagire con una partita, facendo una mossa o inviando un messaggio ai giocatori.

<Interface> LobbyController

Questa interfaccia definisce i metodi con i quali è possibile aggiungere un giocatore ad una partita.

StandardServerController

Questa classe implementa le interfacce `ServerController`, `GamesManagerController` e `AppServer`(si guardi il package `Distributed` per la descrizione di questa interfaccia).

- Per ogni utente collegato al server memorizza l'associazione Client - User in una mappa.
- Per ogni partita creata memorizza l'associazione `GameId` - `GameController` in una mappa.

StandardGameController

Questa classe implementa le interfacce `GameController` e `LobbyController`.

- Per ogni giocatore partecipante memorizza l'associazione Client - Player in una mappa.
- Possiede e gestisce il model di un Game secondo la logica definita nella sua implementazione.

ModelView

Questo package contiene le classi che definiscono “pacchetti di informazioni” riguardanti lo stato di oggetti del Model.

GameInfo

Questa classe contiene informazioni riguardo lo stato della partita (se è iniziata, di chi è il turno, se è l'ultimo round di turni).

ShelfInfo

Questa classe contiene informazioni riguardo lo stato della Shelf di un giocatore.

LivingRoomInfo

Questa classe contiene informazioni riguardo lo stato della board della LivingRoom.

UserInfo

Questa classe contiene informazioni riguardo lo stato di un utente collegato al server (se è connesso ad un Game e gli eventuali errori riportati durante la connessione ad un Game).

PlayerInfo

Questa classe contiene informazioni riguardo lo stato di un giocatore collegato ad una partita; contiene informazioni riguardo eventuali `CommonGoal` conseguiti e riguardo eventuali errori riportati durante l'esecuzione di una azione all'interno della partita (effettuare una mossa o mandare un messaggio).

PlayerChatInfo

Questa classe contiene informazioni riguardo lo stato della chat di un giocatore (insieme dei messaggi mandati o ricevuti dal giocatore).

PersonalGoalInfo

Questa classe contiene informazioni riguardo la descrizione e lo stato di conseguimento di un PersonalGoal di un Player

CommonGoalInfo

Questa classe contiene informazioni riguardo la descrizione e lo stato di un CommonGoal.

PlayerMoveInfo

Questa classe contiene informazioni riguardo una mossa effettuata da un giocatore, contiene una lista di Tile scelte dalla board e l'indicazione della colonna nella quale le si vuole mettere.

PickedTile

Questa classe contiene informazioni riguardo la posizione di una Tile della Board della LivingRoom scelta dal giocatore durante una mossa.

LoginInfo

Questa classe contiene informazioni riguardo una richiesta di un User di diventare Player di un Game.

NewGameInfo

Questa classe contiene informazioni riguardo una richiesta di un User di creare un nuovo Game.

Distributed

Questo package contiene le classi e le interfacce che verranno utilizzate per la comunicazione client-server via socket e RMI. Il package è stato progettato affinché vi sia una totale trasparenza dello strato di Rete da parte delle componenti MVC (in particolare per quanto riguarda la comunicazione via Socket).

<interface> AppServer

Questa interfaccia contiene i metodi utilizzati per aprire e chiudere connessioni via socket.

<Interface> ServerInterface

Questa interfaccia estende tutte le interfacce delle componenti del server delegate alla gestione delle interazioni client → server, contiene quindi tutti e soli i metodi esposti dal server, grazie ai quali il client può interagire con esso. Nel nostro caso estende Remote, ServerController, GameController.

<Interface> ClientInterface

Questa interfaccia estende tutte le interfacce delle componenti del client delegate alla gestione delle interazioni server → client, contiene quindi tutti e soli i metodi esposti dal client, grazie ai quali il server può interagire con esso. Nel nostro caso estenderà tutte le interfacce delle view pronte a ricevere oggetti di ModelAndView

<Interface> SocketObject

Questa interfaccia definisce il metodo con il quale è possibile comunicare tra Client e Server via Socket. Rappresenta a tutti gli effetti un Callback creato ad hoc dal mittente contenente informazioni e un metodo che utilizza tali informazioni per far sì che l'oggetto *sender* interagisca con l'oggetto *receiver*.

Nel nostro caso le informazioni sono i “pacchetti” rappresentati dagli oggetti del ModelAndView e il metodo contiene la chiamata a funzione da eseguire sulla ClientInterface o sulla ServerInterface che utilizza tale oggetto della ModelAndView.

ClientEndPoint

Questa classe estende UnicastRemoteObject e implementa l'interfaccia ClientInterface. E' un oggetto (all'occorrenza remoto) il cui scopo è quello di essere attore sul client delle richieste del server.

ServerEndPoint

Questa classe estende UnicastRemoteObject e implementa l'interfaccia ServerInterface. E' un oggetto (all'occorrenza remoto) il cui scopo è quello di essere attore sul server delle richieste di un client. Ad ogni client fisico è associato un ServerEndpoint.

ServerSocketHandler

Questa classe rappresenta un Client all'interno del Server (estende per questo motivo l'interfaccia ClientInterface).

Il suo scopo è quello di “fingersi un client” attraverso socket:

- traducendo interazioni server→client in SocketObject e inviandoli attraverso un socket.
- ricevendo SocketObject attraverso un socket e traducendoli in interazioni client→server.

ClientSocketHandler

Questa classe rappresenta un Server all'interno del Client (estende per questo motivo l'interfaccia `ServerInterface` e `AppServer`).

Il suo scopo è quello di "fingersi un client" attraverso socket:

- traducendo interazioni client→server in `SocketObject` e inviandoli attraverso un socket.
- ricevendo `SocketObject` attraverso un socket e traducendoli in interazioni server→client.

Funzionamento del Network

Per convenire al design e alle necessità di MVC lo strato di rete è stato progettato **non considerando scambi sincroni di informazioni**, ma **non rendendone incompatibile una possibile implementazione** (la sincronizzazione con RMI è auto-implementata, per quanto riguarda socket è necessario giocare con la sincronizzazione delle funzioni *`send(...)`* e *`waitForReceive(...)`* sia lato server che client).

Al momento della connessione il client genera un `ClientEndpoint` e ne passa un riferimento al server, il quale genererà un `ServerEndpoint` e ne restituirà un riferimento:

- Con RMI vengono scambiati Stub di oggetti remoti (`ClientEndpoint` e `ServerEndpoint`)
- Con Socket vengono generati anche un `ClientSocketHandler` e un `ServerSocketHandler`

Per ogni interazione vengono eseguite chiamate unidirezionali (senza valori di ritorno) sulle interfacce `ServerInterface` e `ClientInterface`. Eventuali valori di ritorno sono inviati asincronamente tramite eventi (prodotti nel nostro caso solo dal model).

Al momento della disconnessione, indipendentemente se richiesta dal client o dal server, si tenta prima l'invio di un segnale di disconnessione e successivamente si chiude unilateralmente il canale di comunicazione.